❸ Defines which version, distribution, and cache type to use

❹ Scans the codebase for vulnerabilities using grype

❺ Assigns an identifier to the current step so that it can be referenced from subsequent steps

❻ The path to the checked-out repository

❼ Whether to fail the build in the event of security vulnerabilities

❽ The minimum security category to be considered as an error (low, medium, high, critical)

❾ Whether to enable the generation of a report after the scan is completed

❿ Uploads the security vulnerability report to GitHub (SARIF format)

Uploads the report even if the previous step fails

Fetches the report from the output of the previous step

Ensures the Gradle wrapper is executable, solving Windows incompatibilities

Runs the Gradle build task, which compiles the codebase and runs unit and integration tests

WARNING The action that uploads the vulnerability report requires the GitHub repository to be public. It works for private repositories only if you have an enterprise subscription. If you prefer keeping your repository private, you'll need to skip the "Upload vulnerability report" step. Throughout the book, I'll assume all the repositories we create on GitHub for the Polar Bookshop project are public.

After completing the declaration of the initial commit stage for the deployment pipeline, commit your changes and push them to the remote GitHub repository. The newly created workflow will be immediately triggered. You can see the execution results on your GitHub repository page on the Actions tab. Figure 3.9 shows an example of the results after running the workflow in listing 3.22. By keeping the result of the commit stage green, you can be quite sure that you haven't broken anything or introduced new regressions (assuming that you have proper tests in place).